System Design Document
for
Recipe Database Management System

Prepared By:

Devika Kundathil Anoop	B240031CS	devika_b240031cs@nitc.ac.in
Jessica Rose Thomas	B240048CS	jessica_b240048cs@nitc.ac.in
Maria Rose Thomas	B240230CS	maria_b240230cs@nitc.ac.in

Instructor:	Dr. A. Sudarshan Chakravarthy
Course:	CS2011E Database Management Systems
Date:	24/03/2026

TABLE OF CONTENTS

1. Purpose……..…...……3

1.1.Document Objective……..……3
1.2.Intended Audience and Document Overview……3
1.3.Definitions, Acronyms and Abbreviations……..……3
1.4.References and Acknowledgments……..……..4

2.Assumptions and Constraints…..……5
2.1.Assumptions……..……...……..……..……5
2.2.Constraints……..……..……..……..……..……..……5

3.Database-Wide Design Decisions……..……..……6
3.1.Behaviour……..……...……..……..……6
3.2.DBMS Platform……..……..…6
3.3.Security Requirements……..……...7
3.4.Performance and Availability Decisions……..7

4.Database Administrative Functions……..……..……..……..8
4.1.Entity Relation Model……..……8
4.2.Relational Schema……...……..9
4.3.Normalization……...……..……..……..11

5.Appendix-SQL Schema (DDL)..……..……...13

1 Purpose

This Software Design Document (SDD) for the Recipe Management System describes the database design chosen based on the system's requirements. It ensures that data is stored correctly and consistently. The Entity-Relationship model built from the system's use cases has been converted into a relational schema for the chosen Database Management System (DBMS).

1.1 Document Objective

The Software Design Document has the following objectives:

1. To describe the design and structure of the Recipe Management System database, including the system components that can be used by both regular users and developers through a DBMS.

2. To serve as a guide for developers during implementation, clearly defining the tables, constraints, and relationships needed to build a working and well-structured database.

1.2 Intended Audience and Document Overview

- This document is intended for individuals involved in the design, development, and deployment of the Recipe Database Management System (RDMS). It is primarily aimed at developers, database designers, project team members and course instructors who are responsible for implementing, maintaining and evaluating the system. In addition, end users such as regular users and administrators can use this document to understand the system’s functionality and usage.

- This document presents a comprehensive overview of the system design and structure. It includes descriptions of the system components, key functionalities, and the underlying database design. The document also outlines the entities, relationships, and schema used to organize recipe data efficiently. It acts as a guide to help stakeholders understand how the system is structured and how different parts of the system interact throughout the development process.

1.3 Definitions, Acronyms and Abbreviations

S.No	Abbreviation / Term	Definition
1	RDMS	Recipe Database Management System
2	SDD	System Design Document
3	SRS	Software Requirements Specification

4	DBMS	Database Management System
5	Admin	System Administrator with full recipe management access
6	Regular User	A registered user who can browse and save recipes
7	API	Application Programming Interface
8	RBAC	Role-Based Access Control
9	PK	Primary Key
10	FK	Foreign Key
11	DDL	Data Definition Language
12	1NF	First Normal Form
13	2NF	Second Normal Form
14	3NF	Third Normal Form

1.4 References and Acknowledgements

- Software Requirements Specification (Group 09, 14/03/2026)
- Fundamentals of Database Systems by Ramez Elmasri and Shamkant Navathe
- IEEE Std 830-1998 – IEEE Recommended Practice for Software Requirements Specifications
- IEEE SRS Template – http://www.cse.msu.edu/~cse870/IEEEXplore-SRS-template.pdf

2 Assumptions and Constraints

2.1 Assumptions

The following assumptions are made while developing the RDMS:

- All recipe, ingredient, category, cooking step, and user data must be stored in a relational database.
- The system will be available 24/7 with minimal downtime.
- Users are assumed to have basic computer literacy, as the system operates through a web-based interface.
- Only administrators have the authority to add, edit, and delete recipes from the system.
- Regular users can browse, search, view, and save recipes to their favourites without any special permissions.
- The database must be updated immediately upon every user action such as registration, adding a recipe, or updating favourites.
- Since the application is web-based, users are assumed to have access to a standard internet browser and a stable internet connection.

2.2 Constraints

The following design and implementation constraints apply to the system:

- The system must be accessible through standard web browsers across multiple devices including desktops, laptops, tablets, and mobile phones.
- Users must authenticate with valid credentials before accessing any system features.
- Role-based access control must be enforced — only admins can add, edit, or delete recipes, while regular users can only browse and manage their favourites.
- The implementation is limited to the chosen technology stack (PostgreSQL, FastAPI and JavaScript).
- The system performance depends on internet connectivity and server availability.
- Personal user data including email and password must be securely stored in compliance with data protection standards.
- Each registered user must have a unique email address for authentication purposes.

3 Database-Wide Design Decisions

3.1 Behaviour

3.1.1 Login or Register

On opening the application, all users are directed to a login page. A registered user enters their email and password to log in. The system detects the user's role and redirects them to a dashboard according to their role. If the user does not have an account, they can register by providing their username, email, and password. After successful registration, they are assigned the regular_user role by default and directed to the recipe browsing dashboard.

3.1.2 Regular User

Regular users, after logging in, are allowed to do the following:

- Browse all available recipes.
- Search and filter recipes by name, ingredient, category, or difficulty level.
- View full recipe details including ingredients with quantities and step-by-step cooking instructions.
- Save recipes to their personal favourites list for quick access later.
- View and manage their favourites list by adding or removing recipes.

3.1.3 Admin

Admins, after logging in, have full control over the recipe content of the system. They are able to:

- Browse, search, and filter all available recipes by name, ingredient, category, or difficulty level.
- Add new recipes with all relevant details including title, description, cooking time, difficulty, category, ingredients with quantities, and step-by-step instructions.
- Edit or delete existing recipes as needed. Upon deletion, all associated steps and ingredient entries are automatically removed.
- Manage the categories and ingredients available in the system.

3.2 DBMS Platform

RDMS is a web-based application designed to provide users with an efficient and interactive platform for managing and exploring recipes. The design architecture includes:

- **Frontend:** HTML5, CSS3, and JavaScript are used to build a responsive and user-friendly interface. The frontend allows users to browse recipes, view detailed information, search using filters, and manage favourites.
- **Backend:** FastAPI (Python) is used as the backend framework. It handles client requests, manages API endpoints, performs input validation, and enforces authentication and role-based access control (RBAC) for users and administrators.
- **Database:** PostgreSQL is used as the relational database management system to store and organize all data related to users, recipes, ingredients, categories, and cooking steps.

The system supports user authentication through a secure login mechanism. Based on user roles, different functionalities are provided: regular users can browse, search, and save recipes, while administrators have additional privileges such as adding, editing, and deleting recipes.

3.3 Security Requirements

- Role-Based Access Control (RBAC) will be enforced through FastAPI middleware, ensuring only admins can add, edit, or delete recipes while regular users are restricted to browsing and managing their favourites.
- User authentication is required before accessing any system features, with separate login flows for admins and regular users.
- Communication between the frontend and backend will be secured using HTTPS to protect data in transit.

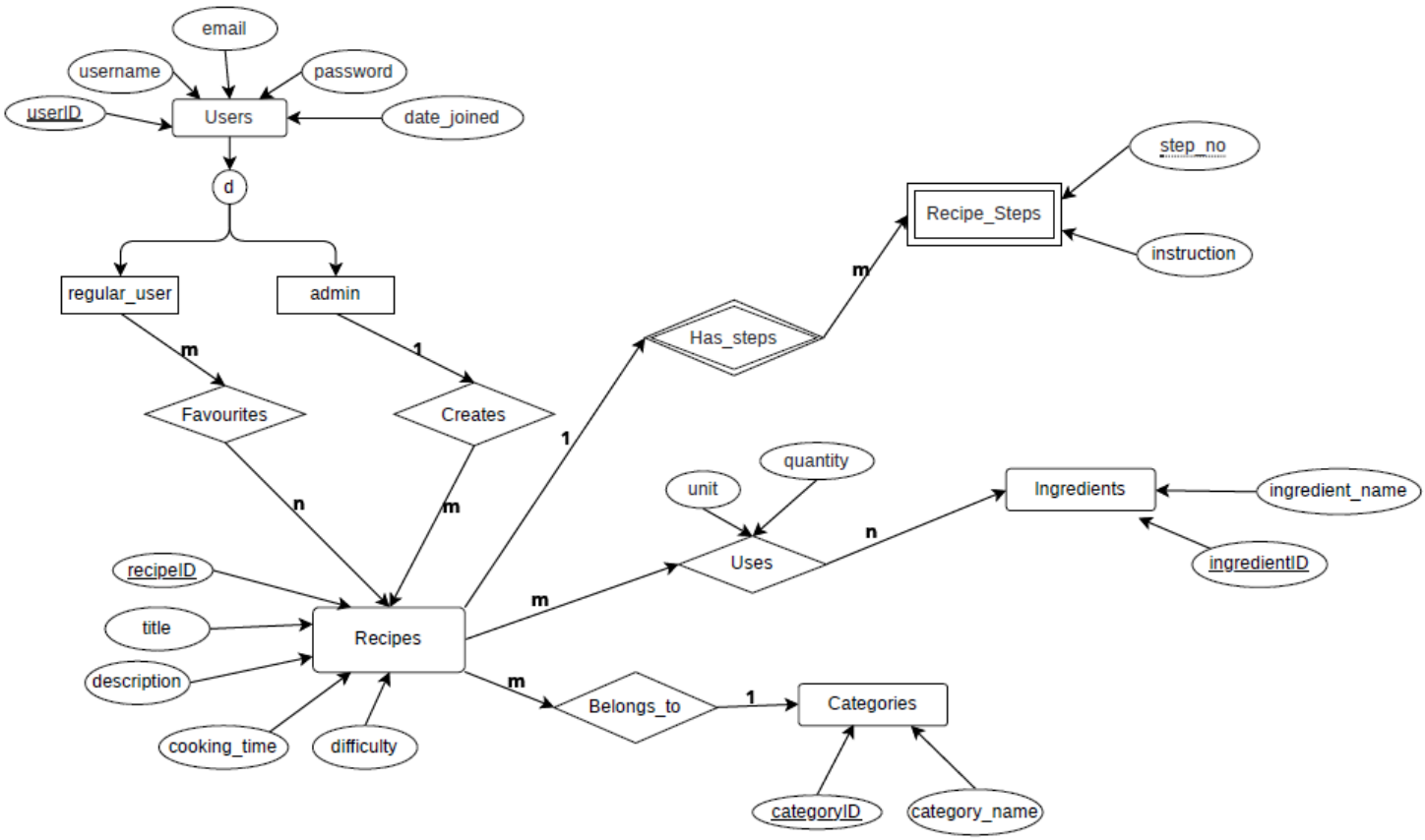
3.4 Performance and Availability Decisions

- The system provides fast response times for user operations such as login, recipe search, viewing recipe details, and managing favourites.
- Most user requests (browsing recipes, filtering, searching) should be processed within a few seconds under normal usage conditions.
- The system shall support multiple users accessing and interacting with the application simultaneously without noticeable performance degradation.
- The system is intended to be available at all times during normal usage, with minimal downtime required for maintenance or updates.
- In case of failures, basic backup and recovery mechanisms shall be in place to prevent data loss and allow restoration of the database.
- Logging mechanisms shall be implemented to monitor system activity and help identify errors or performance issues.
- A stable internet connection is required for users to access and interact with the system effectively

4 Database Administrative Functions

4.1 Entity-Relation Model

The ER model below shows how the Recipe Management System is structured at the database level. It shows the core entities, their attributes, and how they relate to each other. The role-based distinction between admins and regular users is also reflected in the model. Primary key attributes are underlined and weak entities are represented with a double rectangle.



4.2 Relational Schema

The following tables describe the complete schema with the column names, along with its datatypes and constraints for each relation in the Recipe Database Management System (RDMS). The primary keys and foreign keys are also written for each table below.

Table Descriptions:

1.Users

Column	Type	Description
userID (PK)	INT	Unique identifier for each user
username	VARCHAR(50)	User's display name
email	VARCHAR(100)	User's email address (unique)
password	VARCHAR(255)	Hashed password
role	VARCHAR(20)	Role of user (admin / regular_user)
date_joined	DATE	Date the user registered

2. Recipes

Column	Type	Description
recipeID (PK)	INT	Unique identifier for each recipe
title	VARCHAR(100)	Name of the recipe
description	TEXT	Short description of the recipe
cooking_time	INT	Time required (in minutes)
difficulty	VARCHAR(10)	Difficulty level (Easy / Medium / Hard)
categoryID (FK)	INT	References category of the recipe
userID (FK)	INT	References the admin who created it

3. Categories

Column	Type	Description
categoryID (PK)	INT	Unique identifier for category
category_name	VARCHAR(50)	Name of the category

4. Ingredients

Column	Type	Description
ingredientID (PK)	INT	Unique identifier for ingredient
ingredient_name	VARCHAR(100)	Name of the ingredient

5. Recipe_Steps

Column	Type	Description
recipeID (PK, FK)	INT	References the recipe
step_no (PK)	INT	Order of the step in the recipe
instruction	TEXT	Cooking instructions

6. Recipe_Ingredients

Column	Type	Description
recipeID (PK, FK)	INT	References recipe
ingredientID (PK, FK)	INT	References ingredient
quantity	FLOAT	Amount required
unit	VARCHAR(20)	Unit (grams, tbsp, cups, etc.)

7. Favourites

Column	Type	Description
userID (PK, FK)	INT	References user
recipeID (PK, FK)	INT	References recipe

4.3 Normalization

First Normal Form (1NF):

- All tables have a primary key to uniquely identify each record.
- Each column contains atomic, indivisible values — no repeating groups or arrays.
- There are no duplicate rows in any table.
- Example:
  - In the Users table, each user has a unique userID, and columns like username and email store single atomic values.
  - In the Recipes table, attributes like cooking_time and difficulty are atomic single values per recipe. Ingredients are not stored inside the Recipes table but are separated into the Recipe_Ingredients table.

Thus, all tables satisfy the requirements of 1NF.

Second Normal Form (2NF):

- The database is already in 1NF.
- All non-key attributes are fully functionally dependent on the entire primary key.
- For tables with composite keys (Recipe_Ingredients, Recipe_Steps, Favourites), there are no partial dependencies:
  - In Recipe_Ingredients, quantity and unit depend on both recipeID and ingredientID together they are meaningless without knowing which ingredient belongs to which recipe.
  - In Recipe_Steps, instruction depends on both recipeID and step_no, a step cannot exist independently of its recipe.
  - In Favourites, the relationship between userID and recipeID forms the composite key, and no non-key attribute depends on only part of this key.
- All single-key tables (Users, Recipes, Categories, Ingredients) trivially satisfy 2NF.

Thus, all tables meet the criteria for 2NF.

Third Normal Form (3NF):

- The database is already in 2NF.
- There are no transitive dependencies in any table:

- In the Recipes table, attributes like cooking_time, difficulty, and description depend directly on recipeID and not on any other non-key attribute. Category details are stored separately in the Categories table and referenced via a foreign key, avoiding redundancy.
- In the Users table, attributes like username, email, and password depend directly on userID and not on each other.
- In the Ingredients table, ingredient_name depends directly on ingredientID and not on any other attribute.

Thus, all tables satisfy the requirements of 3NF.

Appendix - SQL Schema (DDL)

Users Table

```
CREATE TABLE Users (
    userID INT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    role VARCHAR(20) CHECK (role IN ('admin', 'regular_user')) NOT NULL,
    date_joined DATE NOT NULL
);
```

Categories Table

```
CREATE TABLE Categories (
    categoryID INT PRIMARY KEY,
    category_name VARCHAR(50) NOT NULL UNIQUE
);
```

Recipes Table

```
CREATE TABLE Recipes (
    recipeID INT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    description TEXT,
    cooking_time INT NOT NULL,
    difficulty VARCHAR(10) CHECK (difficulty IN ('Easy', 'Medium', 'Hard')) NOT NULL,
```

```
    categoryID INT,
    userID INT,
    FOREIGN KEY (categoryID) REFERENCES Categories(categoryID) ON DELETE SET NULL,
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE SET NULL
);
```

Ingredients Table

```
CREATE TABLE Ingredients (
    ingredientID INT PRIMARY KEY,
    ingredient_name VARCHAR(100) NOT NULL UNIQUE
);
```

Recipe_Steps Table

```
CREATE TABLE Recipe_Steps (
    recipeID INT,
    step_no INT,
    instruction TEXT NOT NULL,
    PRIMARY KEY (recipeID, step_no),
    FOREIGN KEY (recipeID) REFERENCES Recipes(recipeID) ON DELETE CASCADE
);
```

Recipe_Ingredients Table (junction table for Uses relationship)

```
CREATE TABLE Recipe_Ingredients (
    recipeID INT,
    ingredientID INT,
    quantity FLOAT NOT NULL,
```

```
    unit VARCHAR(20) NOT NULL,
    PRIMARY KEY (recipeID, ingredientID),
    FOREIGN KEY (recipeID) REFERENCES Recipes(recipeID) ON DELETE CASCADE,
    FOREIGN KEY (ingredientID) REFERENCES Ingredients(ingredientID) ON DELETE
CASCADE
);
```

Favourites Table (junction table for Favourites Relationship)

```
CREATE TABLE Favourites (
    userID INT,
    recipeID INT,
    PRIMARY KEY (userID, recipeID),
    FOREIGN KEY (userID) REFERENCES Users(userID) ON DELETE CASCADE,
    FOREIGN KEY (recipeID) REFERENCES Recipes(recipeID) ON DELETE CASCADE
);
```